Part 3: AI/ML Concept Questions

Machine Learning Basics

1. Explain overfitting and underfitting. How would you spot each from training and validation scores?

Overfitting happens when a model learns the training data too closely, including unnecessary details and noise. In this case, the model performs very well on training data but poorly on new validation data. Underfitting happens when a model is too simple to understand the patterns in the data, so it performs poorly on both training and validation data. We can identify these problems by comparing the training and validation scores. For example, a decision tree that memorizes training data but makes poor predictions on new machine readings is overfitting.

2. What is the bias–variance trade-off? Give one change that lowers bias and one that lowers variance.

Bias means the model makes too many simple assumptions and misses important patterns in the data. Variance means the model changes its predictions too much depending on the training data it receives. Increasing the depth of a decision tree can reduce bias because it allows the model to learn more complex patterns. However, reducing the tree depth or applying pruning can reduce variance and help prevent overfitting. We need to find a balance so the model performs well on both training and unseen data.

3. Why do we keep a test set separate from the validation set? What goes wrong if we tune on the test set?

We use the validation set to compare different models and adjust their settings during development. The test set is kept separate to check the final model on data it has never seen before. If we repeatedly use the test set to change the model, the model indirectly starts learning from that data. This can make the final performance score look better than it actually is. For example, we should not keep changing a machine failure model just to improve its test accuracy.

4. What is data leakage? Give two examples, one of them from time-series or sensor data.

Data leakage happens when a model gets information during training that would not normally be available when making real predictions. For example, if we include a machine's failure-type column while predicting whether the machine will fail, the model may already know the answer. Another example is randomly splitting time-series sensor readings, where future readings can enter the training data while earlier readings are used for testing. This gives misleading results because the model gets access to information from the future. To avoid leakage, we should remove unavailable information and split time-based data in chronological order.

5. When would you choose precision over recall, and when recall over precision? Use a tractor or healthcare example.

Precision is important when we want to avoid too many false alarms. For example, if a tractor sends a fault alert and every alert requires an expensive technician visit, we want most alerts to be correct. Recall is important when missing a real problem is more dangerous than receiving a few extra alerts. In healthcare screening, high recall is useful because we want to identify as many people with a possible health condition as possible. The final choice depends on which mistake is more costly.

6. Compare random forest and gradient boosting (e.g. XGBoost). When would you pick each?

Random forest builds multiple decision trees independently and combines their predictions to get a final result. It is generally a good starting model because it is reliable and does not require too much tuning. Gradient boosting builds trees one after another, with each new tree trying to correct the mistakes made by previous trees. XGBoost is a popular implementation of gradient boosting that also includes techniques to improve performance and control overfitting. I would start with random forest for a simple baseline and try XGBoost when I need to improve performance on structured data.

7. Why should time-series data not be split randomly into train and test? What would you do instead?

Time-series data should not usually be split randomly because the order of events is important. A random split might put future readings in the training set and older readings in the test set. This does not represent how a model will work when predicting future events in real life. Instead, I would train the model using older data, validate it using a later period and test it using the most recent data. For example, I could train using January to June readings and test using July readings.

8. A model scores 98% accuracy in testing but performs badly a month after deployment. Give three possible causes.

One possible reason is data drift, where machine operating conditions or sensor readings change over time. A second reason could be data leakage or an unrealistic test dataset that made the model appear more accurate than it really was. A third reason could be a problem in the real production system, such as missing sensor values, incorrect units or different data preprocessing. I would check the incoming data, compare it with the training data and investigate the model's recent prediction errors. This would help identify why the model's performance has dropped.

Deep Learning and Generative AI

9. In simple terms, how does a neural network learn? Mention the loss, gradients and the learning rate.

A neural network learns by making predictions and comparing them with the actual answers. The difference between the predicted and actual values is measured using a loss function. Backpropagation calculates gradients, which tell the network how its weights should change to reduce the loss. An optimizer updates those weights, and the learning rate controls how big each update should be. This process repeats many times until the model learns useful patterns and improves its predictions.

10. What is an embedding? Why do similar sentences end up close together?

An embedding is a numerical representation of something like a word, sentence or document. It converts text into a list of numbers that a computer can compare and process. Embedding models are trained so that sentences with similar meanings usually have similar numerical representations. For example, "The machine is overheating" and "The machine temperature is too high" should have embeddings that are close to each other. We can use cosine similarity to measure how similar two embeddings are.

11. Explain how a retrieval-augmented generation (RAG) system works, step by step. Where can it fail?

RAG stands for Retrieval-Augmented Generation, and it helps an AI model answer questions using external documents. First, we collect documents and divide them into smaller sections called chunks. Next, we convert those chunks into embeddings and store them in a searchable database. When a user asks a question, the system retrieves relevant chunks and sends them to the language model to generate an answer. RAG can fail if the correct information is not retrieved, the documents are outdated or the model generates an answer that is not supported by the retrieved information.

12. What is a hallucination in a large language model? Give two ways to reduce it.

A hallucination happens when an AI model gives an answer that sounds correct but is actually wrong or made up. One way to reduce it is to connect the model to trusted documents using RAG so it can use real information while answering. Another way is to instruct the model to say that it does not know when there is not enough information. We should also check important answers against reliable sources because these methods cannot completely prevent hallucinations.

13. When would you fine-tune a model instead of using prompting or RAG?

Fine-tuning is useful when we want a model to learn a specific way of performing a task, such as following a particular response format or understanding a specialized type of language. Prompting is a good first choice when we only need to guide the model through instructions. RAG is better when the model needs to answer questions using documents or information that may change regularly. I would first try prompting or RAG and check whether they meet the requirements. If the model still struggles with a repeated task and we have enough good-quality training examples, I would consider fine-tuning.

14. A team wants to use an LLM to explain fault codes to tractor operators. List three risks and how you would test for them.

1. **Incorrect advice:** The model might suggest the wrong repair or an unsafe action. I would test it using fault-code examples reviewed by experienced technicians.
2. **Hallucination:** The model might invent an explanation for a fault code it does not understand. I would test unknown and confusing codes and check whether it admits that it does not have enough information.
3. **Difficult explanations:** The answer might be too technical for tractor operators to understand. I would ask operators to test the system and explain the suggested action in their own words.

The system should use approved manuals and should not replace professional repair instructions or safety procedures.

Practical Judgement

15. You have one week and 200 labelled examples. Describe your approach to building a first model.

First, I would understand the problem and identify exactly what we need the model to predict. Then I would inspect the 200 examples to check missing values, incorrect labels and the number of examples in each class. Since the dataset is small, I would start with a simple model such as logistic regression or a decision tree rather than a complex model. I would evaluate it using suitable validation methods and check precision, recall and the confusion matrix instead of depending only on accuracy. Finally, I would review incorrect predictions and explain the limitations of the model, because 200 examples may not be enough for reliable real-world predictions.

16. How would you monitor a deployed model so you know when it needs retraining?

After deploying a model, I would regularly check whether the incoming data is complete and whether its values are changing over time. I would also monitor the model's predictions and check its accuracy, precision and recall whenever actual results become available. If the machine conditions or sensor readings change significantly, the model may not perform as well as before. In that situation, I would investigate the problem and collect newer data for training. Before deploying an updated model, I would test it properly and keep the previous version available in case we need to roll back.